

NHN 아카데미

Iris project

KDT - 3기

김나은

목차

01. 프로젝트 개요

02. 프로젝트 수행 절차 및 방법

03. 프로젝트 수행 경과

0. 자체 평가 의견

01 프로젝트 개요

▶ 아래 내용이 반드시 포함되도록 작성한다.

1 프로젝트 주제 및 선정 배경, 기획의도

- 프로젝트 주제의 특화 포인트, 기존 유사 서비스와 차별화된 내용 제시

2 프로젝트 내용

- 모델 학습 및 관리: 붓꽃 데이터셋을 활용하여 신경망 모델을 학습하고, 이를 파일로 저장/복구하는 시스템 구축.
- API 추론 구현: 사용자의 입력을 받아 신경망 모델을 통해 결과를 도출하는 Predict API 구현.
- 시각화 인터페이스: Chart.js를 연동하여 모델이 예측한 각 품종별 확률을 막대 그래프로 실시간 표시.

3 활용 장비 및 재료

- 개발환경 : Java 21, Spring Boot 3.x, IntelliJ IDEA
- 핵심 라이브러리: DeepLearning4j (신경망 학습), ND4J (행렬 연산), Chart.js (결과 시각화)

4 프로젝트 구조

Controller, Service, Config, DTO

5 활용방안 및 기대 효과

- 다른 정형 데이터셋으로 교체하여 다양한 분류 문제 해결에 즉각 적용 가능.

02

프로젝트 수행 절차 및 방법

▶ 프로젝트의 사전기획과 프로젝트 수행 및 완료 과정으로 나누어서 작성한다.

- 프로젝트 수행 절차를 도식화하여 제시하거나, 더 효과적으로 전달하는 방법 등이 있다면 수정하여 작성 가능
- 기획 단계에서 도출된 주제와 아이디어를 기반으로 실제 프로젝트를 수행한 세부적인 기간과 활동 내용 작성

구분	기간	활동	비고
총 개발기간	▶ 6/1(월)~6/2(화)	- controller, service, config, dto등과 test Code 작성	-

03

프로젝트 수행 경과



프로젝트 결과물이 도출된 과정을 세부적으로 작성한다.

- 이어지는 예시는 하나의 사례로 간단하게 제시한 것이므로 프로젝트 성격에 따라 보다 자세하게 작성
- 결과를 서술하는 과정에서는 활용된 기술 및 구현 방법, 핵심기능, 구현 결과* 등을 상세히 작성
 - * 예시 : 빅데이터 직종의 경우 정확도 등
- 프로젝트 수행 과정이 잘 드러날 수 있도록 가공 과정부터 활용까지 전체적인 프로세스를 단계별로 작성
- 프로젝트 수행 과정에서의 피드백 내용과 그것을 적용(보완 등)한 내용이 포함되도록 작성
- 프로젝트 수행 결과물을 잘 드러낼 수 있는 자료를 첨부하여 작성

* 첨부 자료 예시 : 결과물 사진, 시연 영상, 구동 화면 등 프로젝트의 우수성이 드러날 수 있는 자료

03

프로젝트 수행 경과

DTO 설계 및 작성



DTO 구조(?)

IrisRequest (사용자가 입력하는 데이터)

- 사용자가 입력한 붓꽃의 수치 정보(꽃받침, 꽃잎의 길이와 너비)를 담은 객체입니다. 데이터의 무결성을 위해 record class를 사용하여 불변 객체로 설계합니다.

IrisResponse (서버가 반환하는 예측 결과)

- 머신러닝 모델이 예측한 품종 이름과 각 품종별 확률 분포를 담은 객체입니다.

- IrisRequest

```
/**
 * 클라이언트로부터 전달받는 붓꽃 특징 데이터
 * 붓꽃(Iris) 예측에 필요한 4가지 특징(Feature)
 * @param sepalLength 꽃받침 길이
 * @param sepalWidth 꽃받침 너비
 * @param petalLength 꽃잎 길이
 * @param petalWidth 꽃잎 너비
 */
// @Value
public record IrisRequest( 8개 사용 위치 ㄴ {naeun912}
    double sepalLength, 2개 사용 위치
    double sepalWidth, 2개 사용 위치
    double petalLength, 2개 사용 위치
    double petalWidth 2개 사용 위치
) {}
```

- IrisResponse

```
/**
 * AI 모델의 예측 결과 및 확률 분포 데이터
 * @param predictedSpecies 예측된 품종명 (ex: Setosa)
 * @param probabilities 각 품종별 확률 (ex: {Setosa 0.95, ...})
 */
// @Value
public record IrisResponse(String predictedSpecies, 8개 사용 위치 ㄴ {naeun912}
    Map<String, Double> probabilities) {} 1개 사용 위치
```

ML 모델 학습 및 저장



ML 모델 학습 및 저장

`initModel()`

- 이 메소드는 처음 서버를 실행 했을 때 무조건 학습 모델이 있는지 체크한 후 파일이 없다면 `trainAndSaveModel()` 메서드를 실행시키는 메서드이다.

`trainAndSaveModel()`

- 위에서 말한 학습시키는 메서드이다.
Java에 기본적으로 있는 `irisDataSetIterator`로 데이터를 불러와 준비한 후, 신경망을 설계한다.
랜덤시드 설정, 기본 활성화 함수, 가중치 초기화 방법,
학습 알고리즘 및 확률, 은닉층 설정, 출력층 설정까지 신경망을 설계해준 후 모델을 초기화 해주고 For문을 돌려 학습시킨다.
그 후 학습이 완료된 모델을 저장하고 로그를 출력한다.

`Predict()`

- 이 메서드는 학습된 모델을 가지고 붓꽃의 품종을 예측해주는 메서드이다.
입력된 값을 `INDArray`(행렬)로 변환한다(자바의 기본 `Double` 데이터를 딥러닝 엔진이 이해할 수 있는 행렬로 변환하는 과정)
모델에 변환한 값을 넣어 결과를 얻은 후 결과를 해석주고,
모델이 반환한 확률 중 가장 높은 값을 가진 품종을 찾아 `irisResponse`에 담아 반환한다.

- IrisModelService

```
@EventListener(ApplicationReadyEvent.class) // 서버 시작 직후 실행되는 메서드로 지정??
public void initModel() throws IOException {
    File modelFile = new File(modelProperties.getModelPath());

    if(modelFile.exists()){
        log.info("학습된 모델이 이미 존재합니다. 자동 학습을 건너뛸니다.");
        return;
    }

    log.info("모델 파일이 없습니다. 학습을 시작합니다...");
    trainAndSaveModel();
}

//학습 시키는 메소드
private void trainAndSaveModel() throws IOException{ 1개 사용 위치 ⚡ {naeun912}
    // 데이터 준비 DataSetIterator
    DataSetIterator trainIter = new IrisDataSetIterator( batch: 150, numExamples: 150);

    // 신경망 설계(?) Configuration
    MultiLayerConfiguration configuration = new NeuralNetConfiguration.Builder()
        .seed(123) Builder
        .activation(Activation.RELU)
        .weightInit(WeightInit.XAVIER)
        .updater(new Sgd( learningRate: 0.1))
        .list() ListBuilder
        .layer(new DenseLayer.Builder().nIn(4).nOut(10).build())
        .layer(new OutputLayer.Builder(LossFunctions.LossFunction.NEGATIVELOGLIKELIHOOD)
            .activation(Activation.SOFTMAX)
            .nIn(10).nOut(3).build())
        .build();

    // 모델 초기화 및 학습
    MultiLayerNetwork model = new MultiLayerNetwork(configuration);
    model.init();

    for(int i = 0; i<300; i++){
        model.fit(trainIter);
    }

    // 모델 저장 Serialization
    ModelSerializer.writeModel(model, new File(modelProperties.getModelPath()), saveUpdater: true);
    log.info("모델 학습 및 저장이 완료되었습니다: {} ", modelProperties.getModelPath());
}
```

```
// 사용자의 입력 데이터를 기반으로 붓꽃 품종을 예측
public IrisResponse predict(IrisRequest request){ 2개 사용 위치 ⚡ {naeun912}
    if(model == null){
        throw new IllegalStateException("모델이 로드되지 않았습니다.");
    }

    // 입력을 INDArray(행렬)로 변환 (1행 4열)
    // 자바의 기본 double 데이터를 딥러닝 엔진이 이해할 수 있는 행렬로 변환
    INDArray input = Nd4j.create(new double[] []{{
        request.sepalLength(),
        request.sepalWidth(),
        request.petalLength(),
        request.petalWidth()
    }});

    // 모델에 값을 넣어 결과를 얻음(Inference)
    INDArray output = model.output(input);

    // 결과 해석 (가장 높은 확률을 가진 인덱스 찾기)
    double[] probabilities = output.toDoubleVector();
    String[] labels = {"Setosa", "Versicolor", "Virginica"};

    Map<String, Double> probMap = new HashMap<>();
    int maxIdx = 0;

    for(int i = 0; i < labels.length; i++){
        probMap.put(labels[i], probabilities[i]);
        if(probabilities[i] > probabilities[maxIdx]){
            maxIdx = i;
        }
    }

    // 모델이 반환한 확률 값 중 가장 높은 값을 가진 품종을 찾아 IrisResponse에 담아 반환한다.
    return new IrisResponse(labels[maxIdx], probMap);
}
```

03

프로젝트 수행 경과

예측 서비스 엔진 개발



예측 서비스 엔진 개발

ModelProperties

- Properties 파일에 iris.model-path를 설정한 후
- ModelProperties 파일에 모델 경로 설정해준다.

```
@ConfigurationProperties(prefix = "iris") 3개 사용 위치 {naeun912}
@Component
@Getter
@Setter
public class ModelProperties {
    private String modelPath;
}
```



application.properties ×

<> index.html

JS iris-predic

```
spring.application.name=spring-ai-iris-project
iris.model-path=model/iris-model.zip
```

MultiLayerNetworkConfig

- 모델 파일을 읽어 Spring Bean으로 등록하는 설정 클래스를 생성 후,

모델이 있는지 없는지 확인한 후 없으면 학습돌리는 메서드인 initModel()을 먼저 호출한 후 , 저장된 모델파일을 읽어와서 객체로 복원한다.(역직렬화)

그리고 서비스에 로드된 모델을 설정해준다.(순환 참조 방지를 위해 수정자 주입을 활용함)

```
@Configuration {naeun912}
@RequiredArgsConstructor
public class MultiLayerNetWorkConfig {
    private final IrisModelService irisModelService;
    private final ModelProperties modelProperties;

    @Bean {naeun912}
    public MultiLayerNetwork multiLayerNetWork() throws IOException{
        // 모델 초기화
        irisModelService.initModel();

        File modelFile = new File(modelProperties.getModelPath());

        MultiLayerNetwork model = null;
        if(modelFile.exists()){
            // 저장된 모델 파일을 읽어와서 객체로 복원(역직렬화)
            model = ModelSerializer.restoreMultiLayerNetwork(modelFile);
        }

        // 서비스에 로드된 모델을 설정해준다(순환 참조 방지를 위해 수정자 주입 활용)
        irisModelService.setModel(model);

        return model;
    }
}
```

03

프로젝트 수행 경과

REST Controller 구현



REST Controller

IrisController

- Rest Controller를 어노테이션으로 붙여 데이터를 반환.
- 아까 service에 구현한 predict 메서드를 불러와서 붓꽃의 품종을 예측하도록 controller에 구현한다.

클라이언트로부터 전달받은 JSON 데이터를
IrisRequest 객체로 매핑하여 처리

IrisViewController

- 데이터를 반환하는 controller가 아니라 view와 연결하여 볼 수 있는 controller 구현
- 기본 경로 "/"에 index.html을 연결시킨다.

```
@RestController // 이 클래스의 모든 응답은 JSON 형태로 반환  Ⓜ {naeun912}
@RequiredArgsConstructor
@RequestMapping(Ⓜ "/api/iris")
public class IrisController {
    private final IrisModelService irisModelService;

    @PostMapping(Ⓜ "/predict")  Ⓜ {naeun912}
    public IrisResponse predict(@RequestBody IrisRequest request){
        // 클라이언트로부터 전달받은 json 데이터를 irisRequest 객체로 매핑하여 처리
        Ⓜ return irisModelService.predict(request);
    }
}

@ControllerⓂ Ⓜ {naeun912}
public class IrisViewController {
    @GetMapping(Ⓜ "/")  Ⓜ {naeun912}
    public String index(){
        return "index";
    }
}
```

03

프로젝트 수행 경과

시각화



시각화 대시보드 구현

index.html

- 세가지 종류의 붓꽃의 예시 데이터를 담아둔 버튼 구현
- 꽃받침 길이와, 꽃받침 너비, 꽃잎 길이, 꽃잎 너비 등을 입력할 수 있는 Input 작성란 구현(기본 값들을 넣어둠)
- JavaScript와 연결시켜 예측하기 버튼 구현 후 예측하기 버튼을 눌렀을 때 예측결과가 chart로 나타나도록 구현.

iris-predict.js

- 사용자가 버튼 클릭 시 fillAndPredict 함수를 통해 입력값을 자동 완성하고, predict 함수를 통해 데이터를 JSON 형태로 서버(API)에 전달.
- fetch API의 async/await를 활용하여 페이지 새로고침 없이 서버로부터 예측 결과를 동적으로 수신.
- Chart.js 라이브러리를 사용하여 서버에서 반환한 확률 분포(probabilities)를 막대 그래프(Bar Chart)로 렌더링.
- chart.destroy() 메소드를 통해 중복 생성되는 차트 객체를 관리하여 메모리 누수 방지 및 최신 데이터 유지.

index.html

```
<h1>Iris Species Classifier</h1>
<div id="input-form">
  <div class="input-group">
    <label>Example Cases:</label>
    <button type="button" onclick="fillAndPredict(6.3, 3.3, 6.0, 2.5)">Virginica</button>
    <button type="button" onclick="fillAndPredict(6.1, 2.8, 4.0, 1.3)">Versicolor</button>
    <button type="button" onclick="fillAndPredict(5.1, 3.5, 1.4, 0.2)">Setosa</button>
  </div>
  <hr>
  <div class="input-group">
    <label>Sepal Length (꽃받침 길이):</label>
    <input type="number" id="sepalLength" value="5.1">
  </div>
  <div class="input-group">
    <label>Sepal Width (꽃받침 너비):</label>
    <input type="number" id="sepalWidth" value="3.5">
  </div>
  <div class="input-group">
    <label>Petal Length (꽃잎 길이):</label>
    <input type="number" id="petalLength" value="1.4">
  </div>
  <div class="input-group">
    <label>Petal Width (꽃잎 너비):</label>
    <input type="number" id="petalWidth" value="0.2">
  </div>
  <button onclick="predict()">예측하기</button>
</div>

<div id="result-area" style="display:none">
  <p class="result-info">예측결과 : <span id="predictedSpecies"></span> </p>
  <div id="chart-container">
    <canvas id="probabilityChart"></canvas>
  </div>
</div>
```

iris-predict.js

```
let chart = null;

function fillAndPredict(s1, s2, s3, s4) {
  document.getElementById('sepalLength').value = s1;
  document.getElementById('sepalWidth').value = s2;
  document.getElementById('petalLength').value = s3;
  document.getElementById('petalWidth').value = s4;
  predict();
}

async function predict() {
  const data = {
    sepalLength: document.getElementById('sepalLength').value,
    sepalWidth: document.getElementById('sepalWidth').value,
    petalLength: document.getElementById('petalLength').value,
    petalWidth: document.getElementById('petalWidth').value
  };

  try {
    const response = await fetch('http://api/iris/predict', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(data)
    });

    const result = await response.json();

    document.getElementById('result-area').style.display = 'block';
    document.getElementById('predictedSpecies').innerText = result.predictedSpecies;
    renderChart(result.probabilities);
  } catch (error) {
    console.error('Error:', error);
    alert('예측 중 오류가 발생했습니다.');
  }
}

function renderChart(probabilities) {
  const ctx = document.getElementById('probabilityChart').getContext('2d');
  const labels = Object.keys(probabilities);
  const values = Object.values(probabilities);
  if (chart) {
    chart.destroy();
  }

  chart = new Chart(ctx, {
    type: 'bar',
    data: {
      labels: labels,
      datasets: [
        {
          label: '꽃종류 확률',
          data: values,
          backgroundColor: [
            'rgb(255, 99, 132, 0.5)',
            'rgb(54, 162, 235, 0.5)',
            'rgb(75, 192, 192, 0.5)'
          ],
          borderColor: [
            'rgb(255, 99, 132, 1)',
            'rgb(54, 162, 235, 1)',
            'rgb(75, 192, 192, 1)'
          ],
          borderWidth: 1
        }
      ]
    },
    options: {
      scales: {
        y: { beginAtZero: true, max: 1 }
      }
    }
  });
}
```



프로젝트 결과물에 대한 프로젝트 기획 의도와의 부합 정도 및 실무 활용 가능 정도, 달성도, 완성도 등 훈련생의 자체적인 평가 의견과 느낀 점을 작성한다.

이번 프로젝트를 통해 자바 환경에서 인공지능 모델을 다루는 기술적 성취를 경험할 수 있었습니다.

처음에는 생소한 기술 스택으로 인해 구현 과정이 막막하기도 했으나, 교재와 학습 자료를 토대로 파이프라인을 체계적으로 설계하며 스스로의 힘으로 시각화 결과물까지 만들어냈다는 점에서 큰 성취감을 느꼈습니다.

특히 프론트엔드와 백엔드의 비동기 통신 구조를 이해하고, 정형 데이터셋을 실제 서비스로 전환하는 과정에서 실무적인 개발 역량을 크게 키울 수 있었습니다.

본 프로젝트에서 구현한 ‘입력-추론-시각화’ 파이프라인은 향후 다른 분류 문제나 실시간 데이터 분석 서비스에도 충분히 응용 가능할 것으로 판단하며, 기술 완성도 측면에서 자가 점수 10점의 만족스러운 결과를 얻었습니다.

이번 경험을 발판 삼아 더욱 복잡하고 실용적인 인공지능 서비스 개발에 도전해 보고자 합니다.